ER Diagram student resources

Resource	Suggestion	Description
University of Cape Town: Entity-Relationship Modelling	pre-lecture/assignment review	Walkthrough of design: identify entities, attributes, relationships, cardinalities, then map the model to relational tables. It covers one-to-one, one-to-many, and many-to-many relationships.
University of Cape Town: Enhanced ER Modelling	optional enrichment	Introduces specialization/generalization, aggregation, and composition.
Database System Concepts slides	optional student reference	Silberschatz–Korth–Sudarshan text provides downloadable material for "Database Design: The Entity-Relationship Approach,"
IBM: What is an Entity Relationship Diagram?	Short introductory reading	Best for students who are completely new to ERDs. It succinctly defines ERDs as a way to visualize database items and their relationships, and introduces the familiar symbols used in ER modeling.

Diagramming resources

- draw.io: Entity-relationship shapes for ER database models
  This is the best official reference for the mechanics of building ERD-style table entities in draw.io: enabling the Entity Relation library, adding rows, handling keys, connectors, and templates.
- draw.io: Create an ERD from SQL
  Students can paste MySQL or SQL Server DDL into draw.io to generate entity shapes that include attributes and primary/foreign keys, then compare that implementation-oriented view with their conceptual ERD. Validation activity after they convert an ER design to tables.
- ERDPlus
  Tool for practice with step-by-step beginner lessons for ER diagrams and relational modeling, along with ERD, relational-schema, and star-schema workspaces.

Example-driven resources

- Redgate: ER Diagram for a University Database
  Models course planning, students, enrollments, rooms, and instructors.
- Creately: University Management System ERD examples
  Visual gallery of alternative ways to model the university domain.

An **ER diagram (ERD)** is a visual design model for describing the business data and how concepts relate. A **database schema** is the technical specification—often implemented in SQL—that defines exactly how that data is stored in a database.

Core distinction

Aspect	ER diagram	Database schema
Main purpose	Understand and communicate the data model	Build, enforce, and query the actual database
Main focus	Real-world entities, attributes, and relationships	Tables, columns, data types, keys, constraints, indexes, and other database objects
Abstraction	Conceptual or logical; higher level	Logical and physical; more detailed and DBMS-specific
Typical audience	Analysts, instructors, stakeholders, designers	Developers, DBAs, and database systems
Usual form	Boxes, relationship lines, cardinality symbols	SQL DDL or formal table definitions
Timing	Usually designed first	Derived from the design and then implemented

Example

Suppose you are designing an online-store database.

In an ER diagram

You might show:

- **Customer** places **Order**
- **Order** contains **Product**
- A customer can place many orders: $1:M$
- An order can contain many products, and a product can appear on many orders: $M:N$

At this point, you are modeling the business rules—not necessarily deciding exact column types or table names.

In a schema

You turn that model into implementation details

```
CREATE TABLE Customer (
  customer_id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL
);

CREATE TABLE Orders (
  order_id INT PRIMARY KEY,
  customer_id INT NOT NULL,
  order_date DATE NOT NULL,
  FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)
);

CREATE TABLE Product (
  product_id INT PRIMARY KEY,
  product_name VARCHAR(200) NOT NULL,
  unit_price DECIMAL(10,2) NOT NULL
);

CREATE TABLE OrderItem (
  order_id INT NOT NULL,
  product_id INT NOT NULL,
  quantity INT NOT NULL,
```

```
  PRIMARY KEY (order_id, product_id),
  FOREIGN KEY (order_id) REFERENCES Orders(order_id),
  FOREIGN KEY (product_id) REFERENCES Product(product_id)
);
```

The schema specifies details the ERD may not: INT, VARCHAR(255), NOT NULL, UNIQUE, primary keys, foreign keys, and the bridge table OrderItem.

How they connect

A common workflow is:

1. Identify business objects: Customer, Order, Product.
2. Draw an ERD showing their attributes and relationships.
3. Convert entities into tables.
4. Convert identifiers into primary keys.
5. Convert relationships into foreign keys—or, for many-to-many relationships, an associative/bridge table.
6. Add database-specific details such as data types, defaults, check constraints, indexes, views, triggers, and permissions.

So, an ERD answers: **“What information does the business need, and how is it related?”**

A schema answers: **“Exactly what database objects and rules will store and protect that information?”**